Step 2 — One Tool, One Loop: the Smallest Possible Working Agent — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: an agent is really just a model running in a loop that can ask to call tools. This step builds the **real** Worker loop — the exact one the finished project uses — but gives it only one pure-logic tool, so every line of the loop can be understood before more tools arrive. The full pattern is here from the start: **think** (a model call) → **act** (run the tool it asked for) → **observe** (feed the result back), plus a step log and a hard step limit. **If not:** Step 3 would have to rewrite the loop to add a limit and a log, instead of just adding tools.

After this step the project runs as a small agent: ask for a temperature in Fahrenheit and the model calls the converter, you see each step, and it can never loop forever. Files from Step 1 that aren't shown here don't change.

The tool

Story — `tools/converter.py`: this is the simplest possible tool — pure maths (a celsius→fahrenheit conversion), no network — so it can't fail for any outside reason. Starting with a tool that can't have an outage means any problem you see really is a loop bug, not a flaky connection. (Also create an empty `tools/__init__.py`.) **If not:** a network problem could look like a loop bug, and you'd chase the wrong thing.

```
# src/tools/converter.py
# The converter tool: celsius_to_fahrenheit(), pure logic with no outside
# calls. Its docstring tells the model when to use it.
from langchain_core.tools import tool

@tool
def celsius_to_fahrenheit(celsius):
    """Convert a temperature from Celsius to Fahrenheit. Use it only when
    you already have a Celsius number and the user wants Fahrenheit."""
    return round(celsius * 9 / 5 + 32, 1)
```

Solution — the finished loop shape, with one tool

Changed from Step 1: `exceptions.py` adds Doc07's `MaxIterationsExceeded`.

Story — `exceptions.py`: this defines `MaxIterationsExceeded`, and crucially it *carries the partial step log* — so when the Worker hits its limit, the error still shows exactly where it got stuck and what it tried. An exception that dropped that context would throw away the most useful debugging information at the worst moment. **If not:** the limit would throw away every step that happened before it, leaving only "gave up."

```
# src/exceptions.py
# The project's named errors: MissingConfigError (Doc01), and
# MaxIterationsExceeded (Doc07), which carries the partial step log.

class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

class MaxIterationsExceeded(Exception):
    def __init__(self, message, steps):
        self.steps = steps
        super().__init__(message)
```

Story — `prompts/worker.txt`: this tells the model *when* to reach for its tool instead of answering from memory. Models will happily convert temperatures themselves — and get them slightly wrong — so the prompt is what actually makes the tool get used. **If not:** the tool would exist but be called only sometimes, and the "verified" answers wouldn't really be verified.

`src/prompts/worker.txt`

```
You are a research assistant. Use your tool for every temperature
conversion — never convert in your head. Answer everything else directly.
```

(Step 3 rewrites this prompt once there are 3 tools.)

Story — `agents/worker.py`: this is the heart of the project — Doc07's Build Task Worker loop driving Doc06's `@tool` tools. `ALL_TOOLS` is the toolbox (one tool for now). `find_tool()` and `run_tool()` turn **any** failure — an unknown tool, bad arguments, or a crash inside the tool — into a plain `Error: ...` observation the model can read and react to, instead of a crash. Every call becomes a step record, and the full log comes back in `AgentResult` (or inside `MaxIterationsExceeded` when the limit is hit). This design is exactly why the file does **not** change in Step 3 except for the tool list — new tools just slot into `ALL_TOOLS`. **If not:** a bad tool call would crash the run, and a wrong answer would have no record of how it was reached.

```
# src/agents/worker.py
# The Worker agent: run_worker() is the ReAct loop — ask the model, run the
```

```
# tool it asks for, send the result back — with a step log and a hard
# step limit. ALL_TOOLS lists the tools it may use.
from dataclasses import dataclass, field

from src.exceptions import MaxIterationsExceeded
from src.tools.converter import celsius_to_fahrenheit
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

ALL_TOOLS = [celsius_to_fahrenheit]

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

def find_tool(tools, name):
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None

def run_tool(call, tools):
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        result = tool.invoke(call["args"])
        return str(result), None
    except Exception as e:
        return None, str(e)

def run_worker(model, task, max_iterations):
    logger = get_logger(__name__)
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [
        {"role": "system", "content": load_prompt("worker")},
        {"role": "user", "content": task},
    ]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)
```

```
        for call in response.tool_calls:
            result, error = run_tool(call, ALL_TOOLS)
            if error is None:
                observation = result
                logger.info("step %s: %s -> %s", step, call["name"], result)
            else:
                observation = f"Error: {error}"
                logger.warning(
                    "step %s: %s failed -> %s", step, call["name"], error
                )
            messages.append({
                "role": "tool",
                "tool_call_id": call["id"],
                "content": observation,
            })
            steps.append({
                "step": step,
                "tool": call["name"],
                "arguments": call["args"],
                "result": result,
                "error": error,
            })

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)
```

Changed from Step 1: `main.py` calls `run_worker()` instead of the chain, prints the step log with `print_steps()`, and catches `MaxIterationsExceeded` and `openai.RateLimitError`.

Story — `main.py`: this makes hitting the step limit end *one task* with its partial log printed, not the whole app — so you can see how far the Worker got and why it stopped, then ask the next question. **If not:** you'd see "gave up" with no clue why, or the whole program would crash out.

```
# main.py
# The file you run: python main.py
# Runs the Worker on each task, prints its step log and the answer, and
# turns each known error into one clear message.
import sys

import openai

from src.agents.worker import run_worker
from src.config import load_config
from src.exceptions import MaxIterationsExceeded, MissingConfigError
from src.models.llm import get_model
```

```
from src.utils.logger import get_logger

def print_steps(steps):
    for step in steps:
        if step["error"] is None:
            outcome = step["result"]
        else:
            outcome = "Error: " + step["error"]
        print(f"  step {step['step']}: {step['tool']}({step['arguments']})")
        print(f"          -> {outcome}")

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    model = get_model(config)
    print("ResearchHand — ask a question, or /quit to exit.")

    while True:
        task = input("You: ").strip()
        if task == "/quit":
            break
        if task == "":
            continue

        try:
            result = run_worker(model, task, 5)
        except MaxIterationsExceeded as e:
            print(f"Gave up: {e}")
            print_steps(e.steps)
            continue
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            logger.warning("Rate limited after the client's retries.")
            print("We're busy right now — wait a moment and try again.")
            continue

        print_steps(result.steps)
        print(f"Answer: {result.final_answer}")
```

```
if __name__ == "__main__":
    main()
```

Story — `tests/unit/test_tools.py` : this checks the tool with no model and no network. Case 2 is the interesting one: it proves `@tool` validates the argument's *type* before your function ever runs — the Pydantic contract from Doc06 — so a bad type is rejected at the door, not deep inside the function. **If not:** a wrong argument type would only show up during a real, paid run.

```
# tests/unit/test_tools.py
# Checks the tools. Each case prints OK or WRONG.
# No API calls, no API key, no network needed.
# Run from the repo root: python -m tests.unit.test_tools
from pydantic import ValidationError

from src.tools.converter import celsius_to_fahrenheit

def test_converter():
    result = "OK"
    if celsius_to_fahrenheit.invoke({"celsius": 18}) != 64.4:
        result = "WRONG"
    print(f"case 1 (18°C is 64.4°F)           -> {result}")

def test_converter_rejects_bad_arguments():
    result = "WRONG"
    try:
        celsius_to_fahrenheit.invoke({"celsius": "hot"})
    except ValidationError:
        result = "OK"
    print(f"case 2 (celsius='hot' refused)    -> {result}")

if __name__ == "__main__":
    test_converter()
    test_converter_rejects_bad_arguments()
```

Story — `tests/unit/test_agents.py` : this checks `find_tool()` and `run_tool()` with hand-made tool calls — no model needed — one function per case: a known tool, an unknown tool, a good call, a call to an unknown tool, and bad arguments. These two functions are the project's safety net (they turn every failure into a readable observation), so they're tested exhaustively and for free. **If not:** a bug in how failures are turned into observations would only appear during a real, paid run.

```
# tests/unit/test_agents.py
# Checks the Worker's helper functions with hand-made tool calls.
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_agents
from src.agents.worker import ALL_TOOLS, find_tool, run_tool
```

```
def test_find_known_tool():
    result = "OK"
    if find_tool(ALL_TOOLS, "celsius_to_fahrenheit") is None:
        result = "WRONG"
    print(f"case 1 (known tool found)          -> {result}")

def test_find_unknown_tool():
    result = "OK"
    if find_tool(ALL_TOOLS, "send_email") is not None:
        result = "WRONG"
    print(f"case 2 (unknown tool gives None)   -> {result}")

def test_run_good_call():
    call = {"name": "celsius_to_fahrenheit", "args": {"celsius": 0}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output != "32.0" or error is not None:
        result = "WRONG"
    print(f"case 3 (good call gives 32.0)      -> {result}")

def test_run_unknown_tool():
    call = {"name": "send_email", "args": {}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output is not None or error is None:
        result = "WRONG"
    print(f"case 4 (unknown tool gives error)  -> {result}")

def test_run_bad_arguments():
    call = {"name": "celsius_to_fahrenheit", "args": {"celsius": "x"}}
    output, error = run_tool(call, ALL_TOOLS)
    result = "OK"
    if output is not None or error is None:
        result = "WRONG"
    print(f"case 5 (bad arguments give error)  -> {result}")

if __name__ == "__main__":
    test_find_known_tool()
    test_find_unknown_tool()
    test_run_good_call()
    test_run_unknown_tool()
    test_run_bad_arguments()
```

Changed from Step 1: `tests/integration/test_live_agents.py` gets cases 2-4 — `tools_used()`, three test functions, their imports, and three calls at the bottom.

Story — `tests/integration/test_live_agents.py`: this runs three Worker cases against the real model: a conversion question uses the tool, a general question uses none, and `max_iterations=1` raises `MaxIterationsExceeded`. Together they turn the project's two core promises — "it only calls the tool when needed" and "it always stops" — into things the suite actually verifies. **If not:** those two guarantees would be claims, not checks.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API (and Open-Meteo) — each step
# adds cases. Each case prints OK or WRONG.
# Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents

# new in Step 2
from src.agents.worker import run_worker
from src.chains.direct_answer import build_direct_chain
from src.config import load_config
# new in Step 2
from src.exceptions import MaxIterationsExceeded
from src.models.llm import get_model

def test_direct_answer(model):
    chain = build_direct_chain(model)
    answer = chain.invoke({"question": "What is the capital of France?"})
    result = "OK"
    if "Paris" not in answer:
        result = "WRONG"
    print(f"case 1 (direct answer)  -> {result}: {answer}")

# new in Step 2
def tools_used(outcome):
    names = []
    for step in outcome.steps:
        names.append(step["tool"])
    return names

# new in Step 2
def test_conversion_uses_tool(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    used = tools_used(run_worker(model, task, 5))
    result = "OK"
    if "celsius_to_fahrenheit" not in used:
        result = "WRONG"
    print(f"case 2 (conversion)     -> {result}: {used}")

# new in Step 2
def test_no_tool_needed(model):
```

```
    outcome = run_worker(model, "What is the capital of France?", 5)
    result = "OK"
    if len(outcome.steps) != 0:
        result = "WRONG"
    print(f"case 3 (no tool needed) -> {result}: {outcome.final_answer}")

# new in Step 2
def test_step_limit(model):
    task = "What is 25 degrees Celsius in Fahrenheit?"
    result = "WRONG"
    try:
        run_worker(model, task, 1)
    except MaxIterationsExceeded:
        result = "OK"
    print(f"case 4 (step limit)     -> {result}")

if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    test_direct_answer(model)
    # new in Step 2
    test_conversion_uses_tool(model)
    test_no_tool_needed(model)
    test_step_limit(model)
```

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_tools
python -m tests.unit.test_agents
python -m tests.integration.test_live_agents
python main.py
```

Expected output of the two new unit tests:

```
case 1 (18°C is 64.4°F)           -> OK
case 2 (celsius='hot' refused)    -> OK
```

```
case 1 (known tool found)          -> OK
case 2 (unknown tool gives None)   -> OK
case 3 (good call gives 32.0)      -> OK
case 4 (unknown tool gives error)  -> OK
case 5 (bad arguments give error)  -> OK
```

Expected output of `test_live_agents` (the model's wording varies):

```
case 1 (direct answer)  -> OK: The capital of France is Paris.
case 2 (conversion)     -> OK: ['celsius_to_fahrenheit']
case 3 (no tool needed) -> OK: The capital of France is Paris.
case 4 (step limit)     -> OK
```

Expected app session (log lines left out; wording varies):

```
You: What is 25 degrees Celsius in Fahrenheit?
  step 0: celsius_to_fahrenheit({'celsius': 25})
          -> 77.0
Answer: 25°C is 77.0°F.
You: What is the capital of France?
Answer: The capital of France is Paris.
```